

The four models

One row per question. One column per model. Green is ours and runs free on our own machines; amber is rented and is the only thing that costs money per email.

ONE 4-CORE SERVER TODAY 9.4M emails a day · 26,800 users. A \$24/month box.	SAME SERVER, QUANTISED 27.4M emails a day · 78,400 users. One afternoon's work; the script is already written.	RAM, ALL THREE MODELS 2.25 GB Measured with everything loaded at once, not estimated.	REACHES A MODEL AT ALL 20% Free rules drop the rest before any model runs. This is why the numbers are small.
---	--	--	--

Full per-model load figures further down. No GPU is required at any size.

	OURS · STAGE 08 1 · The reader xlm-roberta-base	OURS · STAGE 09 2 · The decider LightGBM 4.7.0	OURS · STAGE 10 3 · The name finder xlm-roberta-base	RENTED · STAGES 11 & 06 4 · The writer gemini-flash-latest
ITS ONE JOB	Sorts email into two boxes "needs an answer" or "doesn't"	Decides if it needs YOU using who you actually reply to	Finds names to hide before anything leaves our building	Writes the reply in your own writing style
WHAT GOES IN	The cleaned email text	17 numbers — never the text	The text, after patterns took the card numbers	Placeholders only — [PERSON_1]
WHAT COMES OUT	1 of 2 labels + a confidence	1 number, 0.00–1.00	Which words are names, addresses, phones	A finished email
WHY IT EXISTS	Box 2 is free — we stop there. This is how 90% of mail costs nothing.	"Is it a request" and "does it need <i>you</i> " are different questions.	A regex cannot find a name. Only a model can.	Sounding like one specific human is the one job a huge model wins at.
WHAT KIND	Language model SequenceClassification	Not AI at all 300 decision trees, voting	Language model TokenClassification	Large language model someone else's
PARAMETERS	278,045,186 192M of it is the 250,002-word vocabulary	300 trees zero neural parameters	277,539,953 same network, 113 outputs instead of 2	not published we don't host it, so it doesn't matter
SIZE ON DISK	556 MB ~140 MB after ONNX int8	1 MB	1.11 GB saved at full precision; ~280 MB quantised	—
SPEED PER EMAIL	110 ms → 25 ms as built → ONNX int8	1 ms	120 ms → 30 ms as built → ONNX int8	~2 s network round trip
RUNS ON	Ordinary CPU no GPU, ever	Ordinary CPU	Ordinary CPU no GPU, ever	Google's servers
RUNS ON HOW MUCH MAIL	~20% rules already dropped the rest	~20%	~8% only what gets drafted	~8% every stage above exists to shrink this
TRAINED ON WHAT	199 emails we read and labelled by hand	Your own mailbox. Nobody labels anything — Gmail already recorded what you opened.	A free public dataset, 200,000 texts. No mailbox involved.	Not our problem
TRAINING TIME	~4 min free Kaggle T4 GPU	~2 sec plain CPU	~3 min free Kaggle T4 GPU	—
SHARED, OR ONE PER PERSON?	Shared one file for every user on earth	One per person yours alone, built from your behaviour	Shared identifiers don't differ per person	Shared it's a service, not a file
NEW USER SIGNS UP	Nothing happens. They get the same file.	2 seconds of CPU. Then weekly.	Nothing happens.	Nothing happens.

HOW GOOD IS IT	0.82–0.90 confidence on real emails. Weak on short fragments — the router catches those.	AUC 0.684 below the 0.72–0.79 published ceiling. Data limit, not design.	0.922 on its dataset. On <i>real</i> documents this whole field scores ~0.3.	not measured needs real users rating real drafts
COST PER EMAIL	Free our hardware	Free our hardware	Free our hardware	Pay per call ~350 tokens each. The only bill.
AT 1,000,000 USERS	5.8 cores 20M runs/day, sustained	0.23 cores +3.3 cores to retrain everyone weekly	2.8 cores 8M runs/day, sustained	8M calls/day 2.8 billion tokens/day

Speeds measured on one laptop CPU core. Core counts are **calculated** from 100 emails per user per day, 20% surviving to stage 8, 8% reaching the paid step, at ONNX int8 speeds — sustained average, so provision 3–4× for business-hours peaks.

THE QUESTION SIR WILL ASK

What happens when there are a million users?

ALL THREE OF OUR MODELS

8.8 cores

Total, sustained, for 100 million emails a day. **That is one ordinary server.** Scaling means adding workers, because no email depends on any other.

RETRAINING EVERY USER WEEKLY

3.3 cores

One million personal models, rebuilt every week, on a third of a server. This is why the decider is trees.

THE RENTED WRITER

8M calls

Per day. **This is the entire bill.** Take the price per token from the vendor's page on the day — it changes.

THE LEVER THAT CONTROLS COST

8%

Not the price per token. Every stage above 11 exists to shrink this. Each point it falls is **1 million fewer paid calls a day** at this scale.

Why our models get cheaper per user, not more expensive

They cost hardware, not fees. Doubling users doubles the cores, and cores are cheap and rentable by the hour. There is no per-email charge anywhere in models 1, 2 and 3 — not at a thousand users, not at ten million.

The one number that decides the architecture

Retraining a personal model for every user, every week: **3.3 CPU cores** with decision trees, against **397 GPUs running continuously** with a neural network. That is not an optimisation — it is the difference between a feature that exists and one that does not.

LOAD

How much each model can carry

MODEL	RUNS ON	SPEED	EMAILS / SEC ONE CPU CORE	EMAILS / DAY ONE CPU CORE	USERS PER CORE
1 Reader xlm-roberta-base	20% of mail	110 ms	9	785,455	39,273
quantised, ONNX int8	20%	25 ms	40	3,456,000	172,800
2 Decider LightGBM, 300 trees	20% of mail	1 ms	1,000	86,400,000	4,320,000
3 Name finder xlm-roberta-base	8% of mail	120 ms	8	720,000	90,000
quantised, ONNX int8	8%	30 ms	33	2,880,000	360,000
4 Writer gemini-flash-latest	8% of mail	~2 s	Not our hardware. Capacity is the vendor's rate limit — and the only per-call cost in the system.		

USERS	EMAILS / DAY	SERVICES 4 VCPU / 8 GB, QUANTISED
1,000	100,000	1
10,000	1,000,000	1
100,000	10,000,000	2
1,000,000	100,000,000	13

Which model constrains us, and which never will

The decider will never be a bottleneck — one core carries 4.3 million users of it, because it reads 17 numbers rather than text.

The reader is the bottleneck at 39,273 users per core. That makes quantising it the one obvious optimisation: **4× capacity for one afternoon**, and the conversion script is already written.

What these numbers are, and what they are not

Per-email speeds and memory are **measured on real hardware**. Users-per-core and servers-needed are **calculated** from stated assumptions — 100 emails per user per day, 20% surviving the free rules, 8% reaching the drafting step, 3.5× for business-hours peaks. **None of it is load-tested under real concurrency**. No GPU is required at any size.

IF HE ASKS WHY NOT SOMETHING ELSE

What was rejected, and on what evidence

INSTEAD OF	WE COULD HAVE USED	WHY WE DIDN'T
1 Reader	mDeBERTa-v3	Tried it. Failed twice — trained one epoch then went to NaN. A broken model saves and loads without complaint and predicts one label for everything.
1 Reader	A hosted API	Would work and charge for every email forever — 20 million classification calls a day at a million users. This is the cost the whole architecture exists to avoid.
1 Reader	Something bigger	Measured plateau . 33M→0.931, 44M→0.954, 184M→0.955, 434M→0.961. Ten times the size bought +0.007 .
1 Reader	Something smaller	Measured cliff . Below 66M costs −8.7 F1 . Genuinely too small.
2 Decider	A neural network	397 GPUs to retrain a million users weekly, against 3.3 cores. Arithmetically impossible.
2 Decider	Reusing model 1	Measured: the email's words predict replies at 0.511 — chance is 0.500. Your history predicts at 0.692 . Different question, different model.
3 Name finder	Regex alone	Perfect on card numbers, blind to names — which is the hard part. We use both: patterns first, model second.
3 Name finder	A hosted API	Would mean sending un-redacted text to an outside company <i>in order to decide what to hide from outside companies</i> .
4 Writer	Training our own	Costs more than years of API calls and produces something worse. Classification has a right answer; writing like a specific human does not.
4 Writer	Self-hosted 7B	Genuinely worth revisiting above a few million drafts a month — but it puts GPUs in the serving path, which nothing else here needs. Not yet justified .